

The Internet Was Not Built for Inference

The modern internet was designed to move information, not to produce intelligence. Its core abstractions emerged around asynchronous human-scale workloads: web pages, files, email, search queries, streaming video, cloud storage, and database retrieval. In each case, the architectural assumption is that information can be stored somewhere, requested by a user, transmitted across a network, and consumed with tolerable latency. TCP/IP, DNS, CDNs, hyperscale cloud regions, stateless request-response systems, and centralized storage architectures all fit this world. They are brilliant systems for distributing packets. They are not naturally optimized for continuous machine cognition.

Inference breaks the old internet assumption because it is not primarily a communication workload. It is a computational production workload. A large language model does not simply retrieve information and send it back to the user. It generates tokens through repeated transformations of enormous state: model weights, activations, attention maps, and KV cache. The relevant unit is no longer the webpage, the video segment, or the API response. It is the token, and token production is constrained less by traditional networking logic and more by physics: memory bandwidth, thermal dissipation, propagation delay, synchronization overhead, and energy cost per bit of intelligence produced.

The common phrase “AI is compute-intensive” is true, but incomplete. Frontier intelligence is increasingly both a memory-movement problem and a computational intelligence problem. At the lowest level, the arithmetic operation itself is not the main villain. Traditional matmul, where input vectors are multiplied through weight matrices across many layers, is precisely the kind of parallel operation modern accelerators were built to perform. The real constraint is feeding those arithmetic units quickly enough to keep them from starving. A GPU has two primary resources: compute, measured in floating point operations per second (FLOPS), and memory bandwidth, measured in bytes moved per second. In a perfectly balanced system, compute never waits on memory, and memory never waits on compute. Real inference systems are never that clean. One resource saturates while the other idles, and the bottleneck determines the economics of the system. For those who find this familiar, it has become increasingly public that prefill (what Gavin Baker calls loading the cannon) is compute-bound, and decode (firing the cannon) is memory-bound for LLMs.

Token generation therefore has two very different physical regimes. First, the system processes the input sequence, builds internal representations, and constructs the key-value (KV) cache that allows future tokens to attend to prior context (see Google’s: “Attention Is All You Need” paper). This phase, known as prefill, is compute-heavy because the model can process the input sequence in parallel. Then comes decode, where the system produces output tokens autoregressively (e.g., one at a time). Each new token requires a forward pass through the model, meaning the system must repeatedly bring model weights, activations, and prior state close enough to the arithmetic units to generate the next probability distribution over the vocabulary. The model is not “thinking” in the human sense (at least not yet). It is repeatedly transforming vectors through layers, attention operations, nonlinearities, and output projections until the next token distribution is formed. In Philip Kiely’s *Inference Engineering*, this is described directly: prefill determines time to first token and is compute-bound, while decode determines tokens per second and is memory-bound.

The physics problem is that every token depends on state. In an autoregressive system, the next token is conditioned on the tokens that came before it. That means inference is not a cleanly separable request-response workload like serving a webpage. The system must preserve continuity across the sequence. Attention allows each token to relate to prior tokens, but that relationship has to be represented, stored, accessed, and updated. The KV cache exists because recomputing prior attention state from scratch for every new token would make inference unusably slow. It is an optimization, but it is also a massive memory object. The larger the context, the more state must be stored and moved. The KV cache is built during prefill, used and updated during decode, and typically lives in GPU (HBM) memory by default for speed reasons.

This is where the internet’s original assumptions start to break. The internet was built for packet delivery across loosely coupled endpoints. Inference wants tightly coupled state machines with extreme locality. Moving a packet across the

public internet is acceptable when a human is waiting for a web page. Moving inference state across a physical distance is destructive when machines are recursively producing intelligence at high frequency. Distance creates latency. Electrical and optical signals are bounded by propagation delay. Switches, NICs, routers, retimers, protocol overhead, serialization, and deserialization all add friction. Resistance and switching losses turn motion into heat. Coordination across devices creates synchronization overhead. Inference is punished by every extra hop because every hop increases the physical cost of maintaining state.

This is also why "GPU count" is a shallow way to think about AI infrastructure. The economics are set by how effectively the system converts power, memory bandwidth, interconnect bandwidth, thermal envelope, and software orchestration into useful tokens. An H200 (NVIDIA Hopper 200 architecture), for example, can deliver enormous theoretical compute but still runs into a finite memory bandwidth ceiling. The H200 has 1,979 teraFLOPS of dense FP8 compute, 141 GB of memory, and 4.8 TB/s of memory bandwidth. Using an ops:byte framing, that produces an FP8 ops:byte ratio of ~412 (the machine's ridge point). For the system to be perfectly balanced, it would need to perform roughly 412 floating-point operations for every byte of memory accessed. Any workload with lower arithmetic intensity than that is memory-bound, regardless of how much theoretical compute is available, and decode sits far below the line. This is exactly why the H200 is attractive for inference. It carries the same FP8 compute as the H100 but pairs it with more memory and far more bandwidth (4.8 TB/s vs 3.35), which is precisely what memory-bound decode is starved for. That bandwidth and capacity per watt, and the lower ridge point that comes with it, is what places it on the Pareto frontier for token production.

This is exactly the trap decode falls into. During prefill, weights are loaded once and run against large matrices of input tokens, producing high arithmetic intensity. During decode, those same weights are pulled through again and again to produce a single token at a time, collapsing the intensity. The model has plenty of arithmetic capacity; it simply cannot use it unless the right data arrives fast enough. It is the computational version of a factory line where the machines are powerful but the conveyor belt cannot feed them. The factory does not need more machines first. It needs better material flow.

This is the same logic playing out at every layer of the stack, and it reveals something about the inheritance. None of the inference toolkits, HBM, SRAM, NVLink, NVSwitch, kernel fusion, FlashAttention, PagedAttention, prefix caching, quantization, disaggregation, exist to move state faster across distance. Every one of them exists to *avoid* moving it. They are workarounds for an architecture that assumed motion was cheap. FlashAttention reduces unnecessary reads and writes to memory. Kernel fusion avoids round trips between memory and compute. Quantization shrinks the number of bits that have to move at all. Prefix caching reuses prior KV state instead of recomputing it. Cache-aware routing sends the next request to wherever the relevant state already lives, rather than dragging the state to the request. Disaggregation splits prefill and decode because they punish different parts of the machine. Read together, they are not optimizations bolted onto a sound design. They are an industry rebuilding around locality, the one principle the internet was free to ignore, and inference cannot.

Agentic systems make this even more violent. A normal web application might involve one user action, one request, one response. An agentic application can trigger chains of model calls, tool calls, retrieval steps, database reads, code execution, planning loops, verification passes, and additional model calls. One user action can become hundreds or thousands of inference events. In that world, latency does not merely affect user experience. It compounds through the reasoning graph. A 50 ms penalty repeated across a long chain of machine-speed operations becomes a real economic tax. The public internet was not designed for persistent machine-speed cognitive recursion. It was designed for humans clicking, reading, watching, downloading, and waiting.

This is an architectural inversion. For decades, centralization worked because the benefits of hyperscale outweighed the cost of communication. Compute could live far away because the workload was mostly tolerant of distance. Inference

changes that relationship. If the value is created by producing tokens in real time, and if those tokens depend on massive state objects that are expensive to move, then locality becomes economically dominant. The winning systems are not the biggest clusters. Size does not matter. The systems that place memory, compute, routing, and power closest to the demand pattern while minimizing the physical cost of producing intelligence will win.

The internet was built to distribute information between humans. Inference infrastructure must produce intelligence between machines. Those are different physical problems. The first rewards reach, redundancy, and scale, and tolerates distance as the price of all three. The second rewards memory locality, bandwidth density, low-latency interconnect, and thermal control, and is punished by distance at every hop. The internet is not bad infrastructure. It is the defining architecture of one computational epoch, the era of moving what we already knew, running headlong into the demands of the next, the era of producing what we don't yet. Those epochs do not share the same physics, and they will not share the same infrastructure.